04b Switch

04b Switch

00 Übersicht · Zurück: 04a Schleifen · Weiter: 99 Übungsaufgaben

Switch-Statement

Ähnelt entfernt einer einfachen if-Kaskade: abhängig vom Ausdruck (*expression*) werden die Anweisungen zu einem bestimmten Fall (*case*) ausgeführt.

```
switch (expression) {
    case label1:
        Anweisungsliste
        break;
    case label2:
        Anweisungsliste
        break;
    case label3:
        …
    default:
        Anweisungsliste
        break;
}
…
```

Ablauf

1. Werte `expression` zu einem Wert aus.
2. Vergleiche den Wert mit dem Label des nächsten `case`.
3. Gleich → führe alle Anweisungen im `case` aus, **bis** `break` erreicht wird; fahre danach im Programm fort.
4. Nicht gleich → weiter zum nächsten `case` (zurück zu 3).
5. Passt kein `case`: prüfe, ob es einen `default`-Fall gibt.
6. Falls ja: führe dessen Anweisungen bis `break` aus.
7. Fahre im Programm fort.

Was ist zu beachten?

- `expression` muss zu `int`, `String`, `char` oder `Enum` ausgewertet werden.
- `case`-Labels müssen innerhalb eines `switch` **eindeutig** sein (keine doppelten Werte).
- `case`-Labels müssen **konstante Werte** sein (Zahl, Buchstabe, String).
- `case`-Labels können **zusammengefasst** werden. **Fall-through**: es werden alle `case`s bis zum ersten `break` ausgeführt.

- `default`: Standardbehandlung, **optional**, darf nur **einmal am Ende** stehen, wird durchlaufen, wenn kein anderer `case` gilt.

Beispiel: ausführlich vs. kompakt

```
// ausführlich (ohne Fall-through)
switch (numberOfDay) {
    case 1: System.out.println("Werktag"); break;
    case 2: System.out.println("Werktag"); break;
    // … 3, 4, 5 genauso
    case 0: System.out.println("Wochenende"); break;
    case 6: System.out.println("Wochenende"); break;
    default: System.out.println("kein gültiger Wochentag");
}

// kompakt (Fall-through sinnvoll genutzt)
switch (numberOfDay) {
    case 1, 2, 3, 4, 5:
        System.out.println("Werktag");
        break;
    case 0, 6:
        System.out.println("Wochenende");
        break;
    default:
        System.out.println("kein gültiger Wochentag");
}
```

Switch als Ausdruck mit Zuweisung zu einer Variable

Eine **Switch-Expression** liefert einen Wert, der einer Variable zugewiesen wird. **Strichpunkt nach `}` ist nötig** wegen der Zuweisung.

Pfeil-Schreibweise (State of the Art): **kein Fall-through, kein `break`** mehr notwendig.

```
String day1 = switch (numberOfDay) {
    case 1, 2, 3, 4, 5 -> "Werktag";
    case 0, 6 -> "Wochenende";
    default -> {
        // Pfeil erlaubt die Verwendung von Code-Blöcken.
        System.out.println("Fehler: kein gültiger Wochentag");
        yield "kein gültiger Wochentag";
    }
};  // Strichpunkt hier notwendig wegen Zuweisung
System.out.println(day1);
```

Doppelpunkt-Schreibweise mit `yield` (vergleichbar zum Pfeil, **keine Code-Blöcke** möglich):

```
String day2 = switch (numberOfDay) {
    case 1, 2, 3, 4, 5:
        yield "Werktag";
    case 0, 6:
        yield "Wochenende";
    default:
```

```
        yield "kein gültiger Wochentag";
};  // Strichpunkt hier notwendig wegen Zuweisung
System.out.println(day2);
```

`yield` liefert den Wert des Zweigs als Ergebnis des `switch`-Ausdrucks.

Übung Tage pro Monat

Klasse `SwitchExcercise` mit `main`. Switch-Ausdruck, der für jeden Monat die Anzahl der Tage (31, 30, 28 oder 29 bei Schaltjahr) liefert. Eingabe: Monatskürzel (`JAN`, `FEB`, …) und Jahr; Pfeil-Schreibweise. Lösung der Folie:

```
static void main(String[] args) {
    String month = args[0];
    int year = Integer.parseInt(args[1]);

    int numberOfDays = switch (month) {
        case "JAN", "MAR", "MAI", "JUL", "AUG", "OKT", "DEZ" -> 31;
        case "FEB" -> {
            if (year % 4  0 && year % 100 != 0 || year % 400  0) {
                yield 29;
            }
            yield 28;
        }
        default -> 30;
    };
    System.out.println(numberOfDays);
}
```

Schaltjahr-Bedingung: 04 if-Anweisung

Zusammenfassung der Vorlesung 4

- Ein Programm besteht aus **einfachen** und **zusammengesetzten** Anweisungen.
- Zusammengesetzte Anweisungen = **Kontrollstrukturen**, die den Kontrollfluss steuern.
- Zur Verfügung stehen einfache Anweisungen (Zuweisungen, Methodenaufrufe), bedingte Anweisungen und Schleifen; alle **beliebig tief verschachtelbar**.
- Bedingte Anweisungen und Schleifen werden über **Bedingungen** gesteuert (arithmetische, relationale, logische Operatoren).
- Methoden, bedingte Anweisungen und Schleifen definieren **Blöcke** mit geschweiften Klammern.